Uso de Spring Boot Actuator para monitoreo

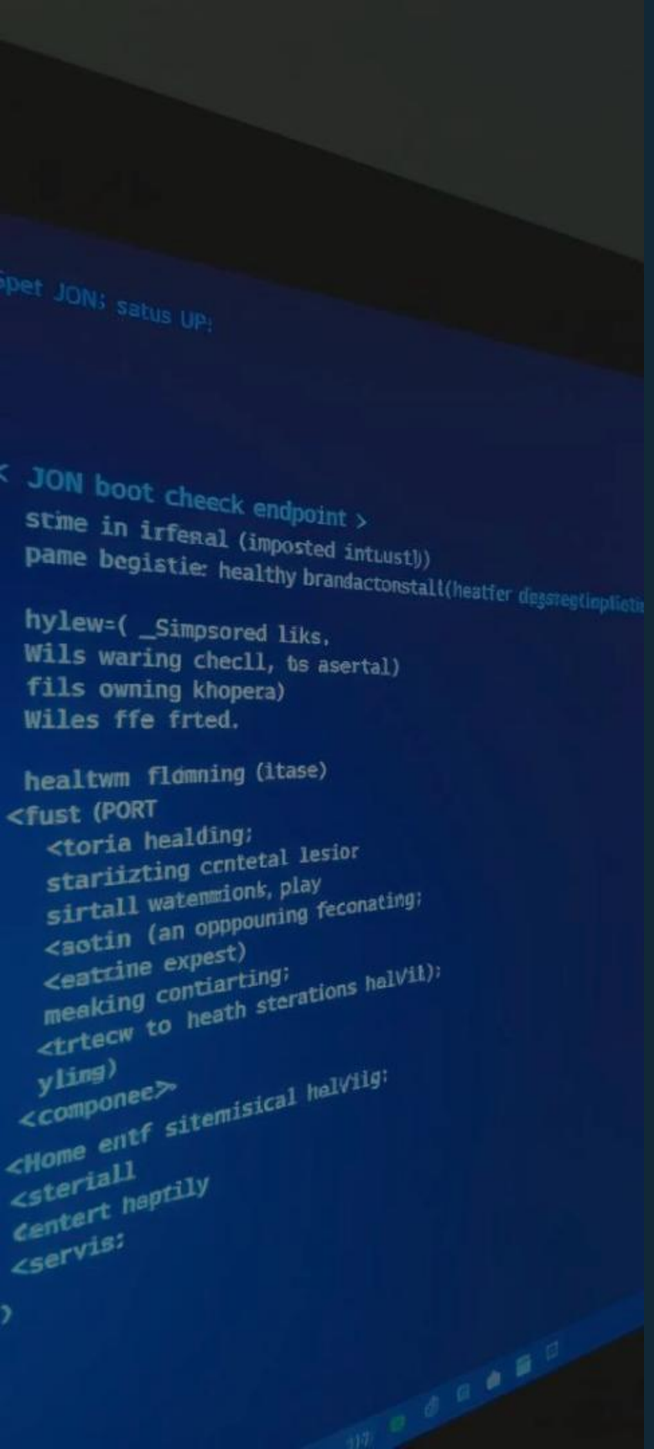
Ricardo Barrios, Victor Villarreal, Carlos Lizarazo, Daniel Ortiz, Alexander Ruidiaz

Observabilidad con Spring Boot Actuator

Spring Boot Actuator proporciona una serie de **endpoints HTTP, JMX y otros canales** para monitorear y gestionar aplicaciones en tiempo de ejecución. Su propósito es **exponer información crítica interna** que ayuda en:

- **Diagnóstico de errores**
- **Monitoreo de estado**
- **Verificación de configuración**
- **Integración con herramientas externas de monitoreo (Prometheus, Grafana, etc.)**

Actuator es esencial para el paradigma de **"operar lo que construyes" (You Build It, You Run It)** en DevOps.



Endpoints más importantes

/actuator/health

Muestra si la app está "UP", "DOWN", "OUT_OF_SERVICE", etc.

Se puede extender con checks personalizados (por ejemplo, conexión a una base de datos externa).

/actuator/metrics

Ofrece **estadísticas detalladas**: CPU, memoria, clases cargadas, GC, requests, etc.

Internamente usa **Micrometer**, una biblioteca de métricas.

/actuator/env

Muestra variables de entorno, propiedades (application.properties, .yml, args, etc.).

Muy útil para validar configuraciones dinámicas en distintos entornos.

Ejemplo de extensión:

```
@Component
public class CustomHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean serviceUp = checkCustomService();
        return serviceUp ? Health.up().build() :
            Health.down().withDetail("error", "Service unreachable").build();
    }
}
```

Más endpoints esenciales

/actuator/metrics

Ejemplos disponibles:

- jvm.memory.used
- http.server.requests
- system.cpu.usage
- process.uptime

Se puede consultar métricas específicas:

GET /actuator/metrics/http.server.requests

/actuator/beans

Lista todos los **beans registrados** en el contexto de Spring.

Se puede ver el tipo, dependencia y ámbito de cada bean. Usado para debugging cuando una dependencia no está bien inyectada o no se cargó un bean esperado.

/actuator/httptrace

Muestra las últimas solicitudes HTTP.

Cada traza contiene: método, URI, duración, status, headers.

Ejemplo de salida:

```
{
  "traces": [
    {
      "request": {
        "method": "GET",
        "uri": "/api/products",
        "headers": { ... }
      },
      "response": {
        "status": 200
      },
      "timeTaken": 42
    }
  ]
}
```

Control de logs en tiempo real

Acceder al endpoint

Navegar a /actuator/loggers para ver todos los loggers configurados

Identificar el logger

Localizar el paquete o clase específica que necesita ajuste

Modificar nivel

Enviar petición POST con el nuevo nivel deseado

Verificar cambios

Comprobar que los logs aparecen con el nivel configurado

Permite **ver y modificar el nivel de logging** de clases específicas en tiempo real.

Cambio dinámico de log level: Muy útil para activar logs solo cuando hay errores sin reiniciar la app.

```
POST /actuator/loggers/com.miapp.servicios
{
  "configuredLevel": "DEBUG"
}
```

Seguridad y personalización

Seguridad con Spring Security

Por defecto, los endpoints críticos están protegidos.
Se puede:

- Definir roles que acceden a /actuator/**
- Usar management.endpoint.*.enabled y management.endpoints.web.exposure.include/exclude para granularidad.

Personalización de Endpoints

Se puede personalizar:

- La ruta base (/actuator)
- El detalle visible
- Agregar nuevos endpoints con @Endpoint

Ejemplo en application.yml:

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, metrics, prometheus
  endpoint:
    health:
      show-details: when_authorized
```

Ejemplo custom endpoint:

```java
@Endpoint(id = "custominfo")
@Component
public class CustomEndpoint {
    @ReadOperation
    public Map customInfo() {
        return Map.of("appStatus", "All systems nominal");
    }
}
```

Accedido desde /actuator/custominfo.

Integración con Prometheus

Agregar dependencia

Incluir micrometer-registry-prometheus en el proyecto:

```
io.micrometer
micrometer-registry-prometheus
```

Activar el endpoint

Configurar la exposición del endpoint en application.yml:

```
management.endpoints.web.exposure.include=prometheus
```

Configurar Prometheus

Definir el scraping de métricas en prometheus.yml:

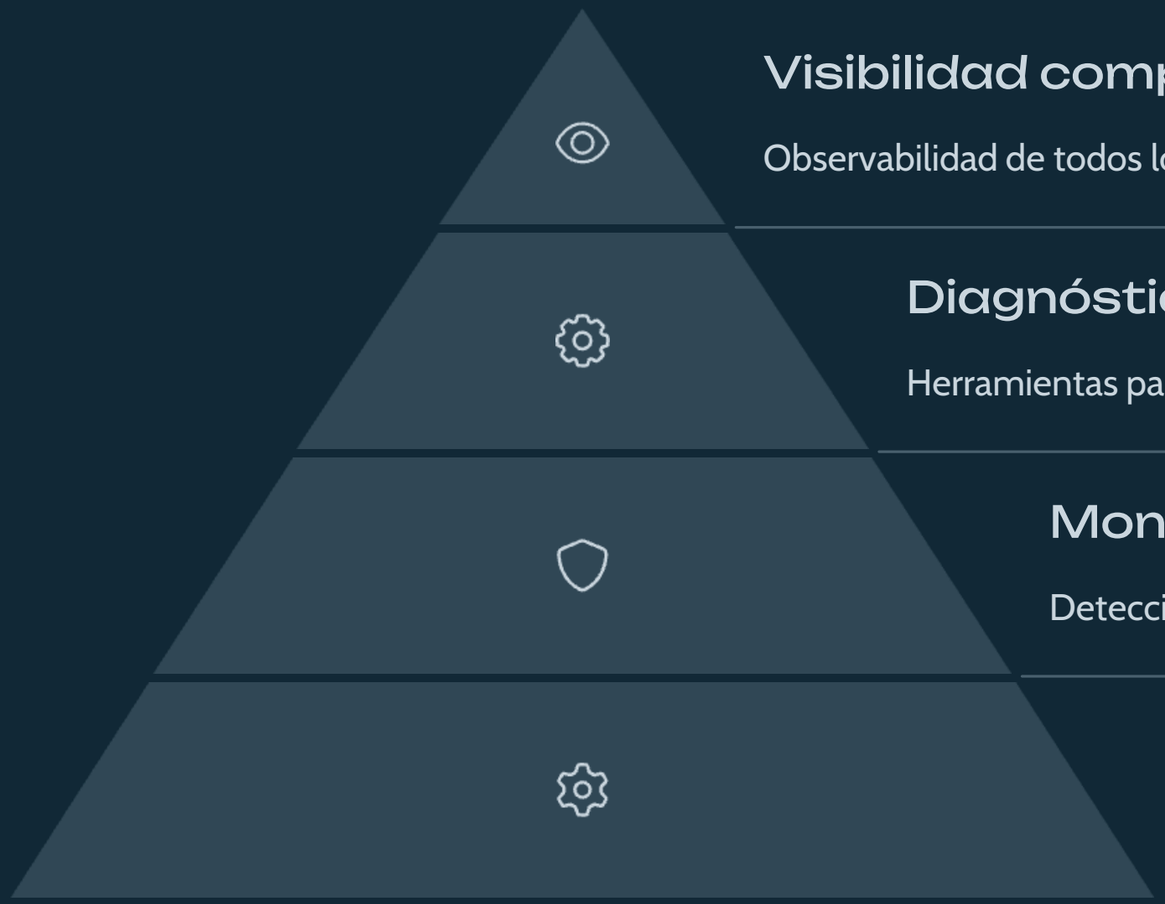
```
scrape_configs:
  - job_name: 'spring-boot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

Conectar Grafana a Prometheus como fuente de datos y crear dashboards

Casos de uso reales para troubleshooting

Escenario	Cómo ayuda Actuator
App lenta o con errores	Usar /metrics y /httptrace para ver cuellos de botella y trazas.
App no arranca correctamente	/health muestra fallos en conexiones de DB, colas, APIs externas.
Configuraciones mal cargadas	/env ayuda a validar qué propiedades están tomando efecto.
¿Qué beans están cargados?	/beans permite auditar el contexto.
Comportamiento inusual	Se activan logs específicos vía /loggers y se hace tracing temporal.

Beneficios de Spring Boot Actuator



Visibilidad completa

Observabilidad de todos los aspectos de la aplicación

Diagnóstico efectivo

Herramientas para identificar y resolver problemas

Monitoreo proactivo

Detección temprana de problemas potenciales

Gestión operativa

Control de la aplicación en tiempo de ejecución

Spring Boot Actuator es una herramienta fundamental para implementar el paradigma de **"operar lo que construyes"** en entornos DevOps, proporcionando los mecanismos necesarios para mantener aplicaciones saludables y responder rápidamente ante incidentes.